Entity Matching with LLMs at Scale: Accuracy, Cost, and Reasoning Effects

Donghao HUANG[1,2], Prasan PAI[2], and Zhaoxia WANG[1] ⋆

[1] School of Computing and Information Systems, Singapore Management University, Singapore, Singapore
dh.huang.2023@engd.smu.edu.sg, zxwang@smu.edu.sg
[2] Research and Development, Mastercard, Arlington, VA, USA
donghao.huang@mastercard.com, prasan.pai@mastercard.com

Abstract. We benchmark large language models for entity matching (EM) using the Compound Entity Matching (ComEM) framework, evaluating 46 model configurations across 8 datasets under a fixed protocol. Beyond accuracy, we quantify *API-equivalent token cost* and analyze the impact of configurable *reasoning/thinking* modes. We find that open-weight models can rival proprietary APIs: GPT-OSS:120b attains 92.10 F1 while costing ∼20× less than GPT-5(high). Reasoning controls have opposite effects across families—low is optimal for GPT-5, whereas high improves open-weight models—and Claude shows the largest sensitivity (Sonnet 4.5: +10.13 F1 with thinking; Opus 4.5: slight degradation at 2.6× cost). Finally, several ultra-budget configurations under $0.10 reach up to 89 F1, enabling inexpensive deployment. Our study offers practical model-selection guidance for EM across diverse budget and infrastructure constraints.

Keywords: Entity Matching · Large Language Models · Entity Resolution · Open-Weight Models · Cost-Effectiveness

1 Introduction

Entity matching (EM), the task of determining whether two data records refer to the same real-world entity, is a fundamental step in entity resolution (ER) [8,10]. EM has broad applications in data integration, data cleansing, and knowledge graph construction. With the emergence of large language models (LLMs), entity matching has shifted toward zero-shot and few-shot paradigms that eliminate the need for extensive labeled training data [15,19,20].

Wang et al. [23] investigated three strategies for LLM-based entity matching—matching, comparing, and selecting—and proposed the Compound Entity Matching (ComEM) framework that composes multiple strategies and LLMs. Their work demonstrated that incorporating record interactions through the selecting strategy yields superior results, and that ComEM achieves both effectiveness and

⋆ Corresponding author: zxwang@smu.edu.sg

cost-efficiency improvements. However, their evaluation was conducted primarily with GPT-3.5 Turbo and GPT-4o Mini, leaving several important questions unanswered.

First, with the rapid evolution of LLMs, it is unclear how newer model generations—including GPT-4.1, GPT-5, and GPT-5.2—perform on entity matching tasks. Second, open-weight models have made remarkable progress, yet their viability for entity matching within the ComEM framework remains unexplored. Third, modern LLMs offer configurable reasoning effort levels (low, medium, high), and the relationship between reasoning effort and entity matching performance has not been studied. Finally, cost-effectiveness across the current model landscape needs systematic evaluation to provide practical deployment guidance.

In this paper, we address these gaps through a comprehensive empirical study. Our contributions are:

- We conduct one of the largest evaluations of LLM-based entity matching to date, testing 46 model configurations across 8 ER benchmark datasets using the ComEM framework, spanning GPT, Claude, and diverse open-weight model families.
- We demonstrate that open-weight models now rival proprietary APIs: GPT-OSS:120b achieves 92.10 F1, outperforming all prior-generation proprietary baselines while costing 20× less than premium models.
- We discover that reasoning effort has opposite effects across model families: low reasoning is optimal for GPT-5, high for open-weight models, and Claude models show the most dramatic thinking impact (+10.13 F1 for Sonnet 4.5, yet −0.25 for Opus 4.5).
- We provide comprehensive cost-performance analysis to guide model selection across diverse budget constraints.

2 Related Work

2.1 Entity Resolution and Matching

Entity resolution has been studied extensively [8,10,17,4]. The blocking-and-matching pipeline is the dominant approach, where blocking filters out obviously dissimilar records and matching identifies duplicates through various techniques. Traditional methods include rule-based [2], distance-based [3], and learning-based approaches [12,14,1]. More recently, pre-trained language models have advanced entity matching [13,22].

2.2 LLM-Based Entity Matching

The advent of LLMs has introduced zero-shot and few-shot paradigms for entity matching [15,19,7,11,20]. Wang et al. [23] conducted a comprehensive investigation of three strategies—matching, comparing, and selecting—and proposed ComEM, which composes strategies and models for improved effectiveness and

cost-efficiency. Their work identified that the selecting strategy, which leverages record interactions by presenting all candidates simultaneously, consistently outperforms pairwise matching.

2.3 Open-Weight and Reasoning-Enhanced LLMs

Recent open-weight models have narrowed the gap with proprietary APIs across many NLP tasks. Models such as DeepSeek [5], Qwen [21], and Llama [6] have demonstrated competitive performance. Additionally, reasoning-enhanced models with configurable thinking effort have emerged as a key trend, with models offering low, medium, and high reasoning settings that trade off computational cost for potential accuracy gains [9]. However, the impact of these developments on entity matching remains largely unexplored.

3 Background: ComEM Framework

3.1 Problem Formulation

Following Wang et al. [23], we formulate entity matching as identifying the matching record from a given anchor record r and its n potential matches $R = \{r_1, r_2, \ldots, r_n\}$ obtained from blocking. This formulation accommodates both single-source deduplication and dual-source record linkage under the one-to-one matching assumption.

3.2 Strategies for LLM-Based Entity Matching

Three strategies have been proposed for applying LLMs to entity matching:

Matching. The LLM acts as a binary classifier, independently determining whether each pair (r, r_i) is a match. This strategy ignores global consistency among record relationships.

Comparing. The LLM compares two candidate records r_i and r_j against anchor record r to determine which is more consistent. This introduces pairwise record interactions but requires $\mathcal{O}(kn)$ LLM invocations.

Selecting. The LLM directly selects the matching record from the candidate list R, leveraging full context of all candidates. This requires only $\mathcal{O}(1)$ LLM invocations per anchor record but is sensitive to position bias in long contexts.

3.3 ComEM: Compound Entity Matching

ComEM [23] addresses the limitations of individual strategies through a two-stage pipeline: (1) *Ranking*—a cost-efficient LLM (Flan-T5-XL) scores all n candidates using the matching strategy, retaining the top-k candidates; (2) *Selection*—a more capable LLM applies the selecting strategy on the filtered k candidates. The selecting prompt supports a "[0] if there is none" option, allowing the model to abstain when no candidate matches. Wang et al. show that k=4 balances F1 and cost, and that this two-stage design mitigates position bias, reduces context length, and improves cost-efficiency.

4 Experimental Setup

4.1 Faithfulness to ComEM Protocol

To ensure a fair comparison and isolate the effect of model choice as the sole variable, we replicate the ComEM evaluation protocol exactly. Specifically we inherit: (i) **Datasets and blocking**: the same 8 pyJedAI [16] clean-clean ER datasets with Sparkly [18] blocking at n=10 candidates; (ii) **Sampling**: 400 anchors per dataset (300 with matches, 100 without; random seed 42), yielding 4,000 candidate pairs; (iii) **Prompts**: the identical matching, comparing, and selecting prompt templates from Wang et al. [23] (their Table 5), including the "[0] if there is none" abstention option; (iv) **ComEM pipeline**: Flan-T5-XL as the ranking model with k=4 candidates retained for selection; (v) **Decoding**: temperature = 0 for all models; (vi) **Candidate ordering**: blocking-score order, consistent with Wang et al., so any position bias is identical across models. We do not shuffle candidates because our goal is controlled comparison under the same positional-bias conditions analyzed in ComEM; randomization would introduce an additional variable.

What we change (only): the selecting-stage LLM and, where supported, its reasoning effort or thinking mode. All other pipeline components remain fixed.

4.2 Datasets

The eight datasets span product (AB: Abt-Buy, AG: Amazon-Google, WA: Walmart-Amazon), citation (DA: DBLP-ACM, DS: DBLP-Scholar), and entertainment (IM: IMDB-TMDB, IV: IMDB-TVDB, TT: TMDB-TVDB) domains.

4.3 Models Evaluated

We evaluate an extensive set of 46 model configurations spanning four categories:

Proprietary API models (OpenAI): GPT-4o-mini, GPT-4o, GPT-4.1-nano, GPT-4.1-mini, GPT-4.1, GPT-5-nano (low/medium/high reasoning), GPT-5-mini (low/medium/high), GPT-5 (low/medium/high), GPT-5.2 (low/medium/high).

Proprietary API models (Anthropic): Claude-Haiku-4.5 (with/without thinking), Claude-Sonnet-4.5 (with/without thinking), Claude-Opus-4.5 (with/without thinking), Claude-Sonnet-4.6 (with/without adaptive thinking), Claude-Opus-4.6 (with/without adaptive thinking).

Open-weight models (locally deployed with Ollama): All model names follow the Ollama naming convention (e.g., `gpt-oss:20b`). This category includes GPT-OSS:20b (low/medium/high), GPT-OSS:120b (low/medium/high), GLM-4.7-Flash:q4_K_M, Gemma3:27b, Llama4:Scout, and Granite4:32b. Models are deployed locally on an NVIDIA DGX Spark using Ollama v0.15.6.

Open-weight models (via OpenRouter API): DeepSeek-Chat-v3.1 (with/without reasoning), Kimi-K2-0905, Kimi-K2-Thinking, GLM-5 (with/without thinking),

Qwen3-235b-a22b-2507 (standard and thinking variants), and MiniMax-M2.5. These models are accessed through the OpenRouter API.

Reasoning effort levels (low, medium, high) or thinking modes (enabled, adaptive, disabled) are configured where supported, enabling systematic study of the reasoning–performance trade-off. These are provider-specific controls with differing internal semantics; we study them as practical deployment knobs rather than a standardized compute measure.

4.4 Cost Calculation

We report *API-equivalent token cost*, computed from measured token counts and published per-token prices (as of Feb. 17, 2026). Proprietary models use their providers' pricing; open-weight models—whether locally deployed or accessed via API—use OpenRouter's published unit pricing as a uniform proxy. This differs from Wang et al.'s [23] approach of estimating open-source cost via inference time $\times$ cloud GPU hourly rate; our metric is hardware-independent and comparable *as an API-equivalent proxy* across models. Cost per run is:

$$\text{Cost} = N_{\text{in}} \times P_{\text{in}} + (N_{\text{out}} + N_{\text{reason}}) \times P_{\text{out}}, \tag{1}$$

where N_{in}, N_{out}, and N_{reason} are the numbers of input, output, and reasoning tokens, respectively, and P_{in} and P_{out} are the corresponding per-token prices.

For locally deployed models, we report API-equivalent token cost as a hardware-independent proxy; actual on-prem cost depends on amortization, utilization, and power. All costs in this paper are summed across 8 datasets (3,200 anchors total); to obtain per-dataset cost, divide by 8. Wang et al. [23] report per-dataset cost, so our totals are approximately 8$\times$ larger for comparable models.

4.5 Response Parsing

The selecting prompt asks the LLM to return a bracketed index (e.g., `[1]` or `[0]` for no match). We extract the index using the regex `\[(\d+)\]`; unparseable responses (e.g., verbose preamble, refusal, empty output) default to *no match*, penalizing recall rather than inflating match predictions. We apply the same parser and fallback rule to all models. We verified parse-failure rates by scanning all cached responses (Appendix, Table 5): 28 of 46 models achieve 0%. Notable outliers include GPT-5-nano(high) at 30.6%—likely contributing to its anomalously low F1 despite high precision—and GLM-5 at 11–20%. All Claude 4.6 configurations achieve 0%, suggesting that thinking modes do not degrade format compliance. For GPT-5, the invalid rate rises from 0% (low) to 0.3% (medium) to 5.8% (high), indicating that higher reasoning effort can increase verbosity-related failures; we therefore report invalid response rates to contextualize performance under different verbosity regimes.

4.6 Evaluation Metrics

We evaluate all models using the compound strategy, which demonstrated the best overall performance in prior work [23]. We report F1 score on the positive class (matching pairs) and total API-equivalent token cost across all 8 datasets (see Section 4.4).

5 Results and Discussion

5.1 Overall Model Performance with Compound Strategy

Table 1 presents the top 15 models alongside selected baselines. GPT-5 and GPT-5.2 variants hold the top 6 positions (93.17–92.14 F1). GPT-OSS:120b(high) ranks 7th (92.10) as the top open-weight model, and Kimi-K2-Thinking ranks 8th (91.84) as the best non-GPT model. All top-15 models exceed 91 F1, with Claude-Haiku-4.5(think) at rank 15 (91.19) as the best Claude configuration.

5.2 Open-Weight Models Match Proprietary Performance

A central finding of our study is that open-weight models deployed locally now compete with, and in many cases exceed, the performance of prior-generation proprietary models (Table 1).

GPT-OSS models. The `gpt-oss:120b(high)` model achieves 92.10 F1, ranking 7th overall and outperforming prior-generation proprietary baselines including GPT-4.1 (90.36 F1) and GPT-4o (89.58 F1). At $1.20 total cost, it costs 20× less than GPT-5(high) ($24.50) while delivering 99.96% of its performance. The smaller `gpt-oss:20b(high)` achieves 91.36 F1 at only $0.46—outperforming GPT-4.1 while costing 2.8× less.

Kimi-K2-Thinking. From Moonshot AI, `kimi-k2-thinking` achieves 91.84 F1, ranking 8th overall and entering the top 10 as the best non-GPT model. It delivers balanced high performance (90.00 precision, 93.83 recall) at $8.29 total cost, outperforming all GPT-5-mini variants.

MiniMax-M2.5. MiniMax-M2.5 achieves 91.34 F1, ranking 13th with well-balanced precision (90.64) and recall (92.12) at $3.74 total cost—competitive with GPT-OSS:120b(medium) and GPT-4.1-mini.

5.3 Reasoning Effort: Opposite Effects Across Families

A surprising finding is that reasoning effort has opposite effects on different model families. Table 2 summarizes the impact across six model families.

GPT-5 family: low reasoning is optimal. For GPT-5, low reasoning achieves 93.14 F1 at $6.25, virtually identical to medium (93.17 F1 at $12.29) while costing 49% less. High reasoning (92.14 F1 at $24.50) actually degrades performance while costing 4× more. This pattern holds for GPT-5-mini, where low (91.72 F1 at $0.71) outperforms medium (91.46 F1 at $1.28) at 45% lower cost.

Table 1. Top 15 models and selected baselines (compound strategy), ranked by mean F1. Baseline row numbers indicate global rank in Table 4 (Appendix). Cost = total API-equivalent token cost across all 8 datasets.

#	Model	F1	Prec.	Recall	Cost
1	gpt-5(medium)	93.17±6.0	92.04	94.42	$12.29
2	gpt-5(low)	93.14±6.4	92.26	94.12	$6.25
3	gpt-5.2(low)	92.75±5.9	90.58	95.12	$3.43
4	gpt-5.2(high)	92.54±6.1	90.22	95.08	$5.74
5	gpt-5.2(medium)	92.53±6.2	90.33	94.92	$4.52
6	gpt-5(high)	92.14±6.8	92.66	91.71	$24.50
7	gpt-oss:120b(high)	92.10±6.3	89.91	94.46	$1.20
8	kimi-k2-think	91.84±6.4	90.00	93.83	$8.29
9	gpt-5-mini(low)	91.72±6.1	88.64	95.08	$0.71
10	gpt-5-mini(high)	91.62±6.2	88.47	95.08	$2.30
11	gpt-5-mini(medium)	91.46±6.4	88.21	95.04	$1.28
12	gpt-oss:20b(high)	91.36±6.5	89.86	93.00	$0.46
13	minimax-m2.5	91.34±5.7	90.64	92.12	$3.74
14	gpt-oss:120b(medium)	91.26±7.2	90.29	92.33	$0.38
15	claude-haiku-4.5(think)	91.19±6.4	88.63	94.00	$8.64
Selected baselines and notable models					
20	claude-opus-4.6(adapt)	90.13±6.4	86.13	94.62	$19.05
23	claude-sonnet-4.5(think)	89.70±6.4	85.23	94.79	$22.32
24	gpt-4o	89.58±6.1	89.43	89.87	$3.10
31	gpt-4o-mini	85.88±6.7	84.82	87.63	$0.72
40	claude-opus-4.5	80.53±6.5	71.55	92.13	$22.32
41	claude-opus-4.5(think)	80.28±6.9	71.19	92.08	$57.07
44	claude-sonnet-4.5	79.57±6.4	70.01	92.17	$11.92
46	granite4:32b	78.49±9.5	81.89	77.08	$0.09

GPT-5.2 family: reasoning effort has minimal impact. All three reasoning levels achieve nearly identical F1 (∼92.5), with low being the clear value choice at $3.43 versus $5.74 for high.

Open-weight models: high reasoning is critical. In stark contrast, GPT-OSS models show dramatic improvement with higher reasoning effort. For `gpt-oss:120b`, F1 increases from 90.11 (low) to 91.26 (medium) to 92.10 (high)—a 1.99 point range. For `gpt-oss:20b`, the range is even larger at 2.62 points. This suggests that open-weight models benefit more from extended reasoning, possibly because they have less internalized knowledge about entity matching patterns and rely more on explicit reasoning.

Claude models: dramatic and asymmetric thinking effects. The Anthropic Claude family exhibits the most extreme thinking impacts of any family tested. Thinking primarily boosts precision while maintaining recall: Sonnet 4.5 gains +10.13 F1 (precision +15.22), Haiku 4.5 gains +8.55, and Opus 4.6 gains +4.95 with adaptive thinking. Yet Opus 4.5 *loses* 0.25 F1 with thinking at 2.6×

Table 2. Impact of reasoning effort across model families. F1 reported as mean across 8 datasets. "Best" = recommended setting considering both F1 and cost; *medium is marginally higher (93.17 vs. 93.14) but costs 2× more. For models with thinking toggle, "On" and "Off" denote thinking enabled/disabled; "Adapt" denotes adaptive thinking.

Model Family	Low	Medium	High	Best	Δ	Trend
gpt-5	**93.14**	93.17	92.14	Low*	1.03	↓
gpt-5.2	**92.75**	92.53	92.54	Low	0.22	≈
gpt-5-mini	**91.72**	91.46	91.62	Low	0.26	≈
gpt-5-nano	88.38	**89.34**	85.71	Medium	3.63	∩
gpt-oss:120b	90.11	91.26	**92.10**	High	1.99	↑
gpt-oss:20b	88.74	90.87	**91.36**	High	2.62	↑
	Off	**On/Adapt**				
claude-sonnet-4.5	79.57	**89.70**		On	10.13	↑↑
claude-haiku-4.5	82.64	**91.19**		On	8.55	↑↑
claude-opus-4.6	85.18	**90.13**		Adapt	4.95	↑
claude-sonnet-4.6	82.48	82.55		Adapt	0.07	≈
claude-opus-4.5	**80.53**	80.28		Off	0.25	↓
glm-5	79.77	**91.02**		On	11.25	↑↑
qwen3-235b	**86.85**	81.76		Off	5.09	↓
deepseek-v3.1	79.16	**86.52**		On	7.36	↑

cost ($57.07), and Sonnet 4.6 gains only +0.07. Without thinking, all Claude models share high recall (92–93%) but poor precision (70–79%), suggesting they default to predicting "match" without deeper analysis.

Thinking can hurt. Qwen3-235b drops 5.09 F1 with thinking at 34× cost. GLM-5 with thinking achieves 91.02 F1 (vs. 79.77 without), though it remains expensive compared to GLM-4.7-Flash (90.08 F1 at $1.74). DeepSeek-v3.1 gains +7.36 F1 with reasoning, confirming model-dependent effects.

5.4 Cost-Performance Analysis

Figure 1 visualizes the cost–F1 trade-off on a log-scaled cost axis. The Pareto frontier comprises 13 of 46 configurations and is dominated by the GPT ecosystem: six GPT-OSS variants (spanning low to high reasoning at both 20b and 120b scales), GPT-5-mini(low), GPT-5.2(low), and GPT-5 at low and medium reasoning. GPT-4.1-nano also appears at the ultra-budget end alongside Gemma3:27b and Qwen3-235b—the only two non-GPT frontier models. No Claude, GLM, or other model family reaches the frontier. The frontier reveals a steep initial slope—moving from $0.08 to $1.20 lifts F1 from 84–89 to 92—followed by sharply diminishing returns beyond $3.

Table 3 distills these trade-offs into actionable budget tiers. The ultra-budget tier (<$0.10) already achieves 87–89 F1; the budget-to-value range ($0.46–$1.20) jumps to 91–92 F1; and beyond $3 each additional dollar buys less than 0.3 F1

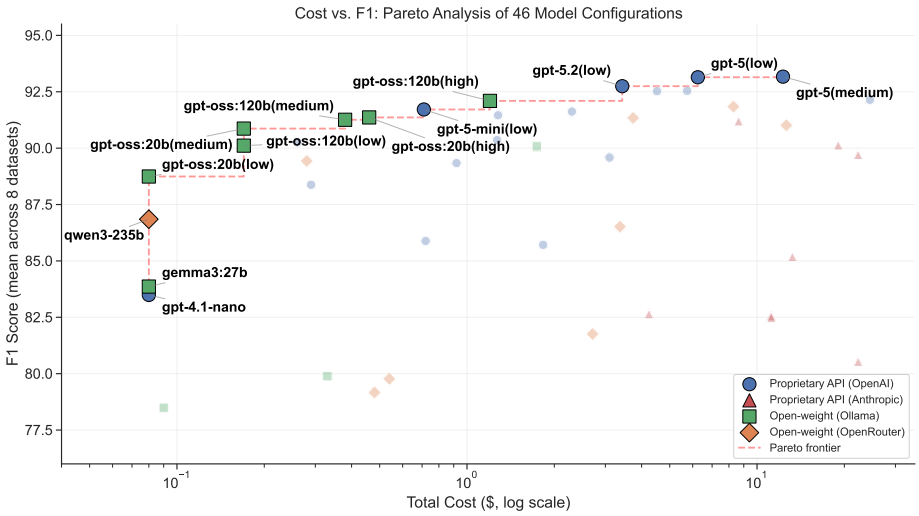

Fig. 1. Cost vs. F1 Pareto analysis of 46 model configurations using the compound strategy. Each point represents a model's mean F1 (across 8 datasets) and total API cost. Red-circled points lie on the Pareto frontier; the dashed red step line traces the frontier. Marker shapes distinguish proprietary API models (circles), open-weight models deployed locally with Ollama (squares), and open-weight models accessed via the OpenRouter API (diamonds).

Table 3. Recommended models by budget tier for entity matching deployment.

Budget Tier	Model	F1	Cost
Ultra-budget	qwen3-235b	86.85	$0.08
Ultra-budget	gpt-oss:20b(low)	88.74	$0.08
Budget	gpt-oss:20b(high)	91.36	$0.46
Value	gpt-5-mini(low)	91.72	$0.71
Value	gpt-oss:120b(h)	92.10	$1.20
Mid-range	gpt-5.2(low)	92.75	$3.43
Premium	gpt-5(low)	93.14	$6.25
Premium	gpt-5(medium)	93.17	$12.29

points, making the mid-range and premium tiers worthwhile only when maximum accuracy is critical.

6 Conclusion

We present one of the largest empirical evaluations of LLM-based entity matching to date, testing 46 model configurations across 8 benchmark datasets. Open-weight models now rival proprietary APIs at dramatically lower costs (GPT-OSS:120b(high): 92.10 F1 at 20× less cost). Reasoning effort has opposite effects across families: low is optimal for GPT-5, high for open-weight models, and Claude models show the most dramatic thinking impact (+10.13 F1 for Sonnet 4.5, yet −0.25 for Opus 4.5). Several ultra-budget models under $0.10 achieve up to 89 F1, democratizing entity matching.

Our study uses clean-clean ER scenarios; performance on dirty data remains to be evaluated. We inherit the ComEM evaluation protocol, which may not reflect all deployment conditions. Model costs change rapidly, so specific comparisons are illustrative. Future work includes dirty-data evaluation, prompt engineering for open-weight models, adaptive reasoning effort selection, and domain-specific entity matching applications.

References

1. Barlaug, N., Gulla, J.A.: Neural networks for entity matching: A survey. vol. 15, pp. 1–37 (2021)
2. Benjelloun, O., Garcia-Molina, H., Menestrina, D., Su, Q., Euijong Whang, S., Widom, J.: Swoosh: A generic approach to entity resolution. vol. 18, pp. 255–276 (2009)
3. Bilenko, M., Mooney, R.J., Cohen, W.W., Ravikumar, P., Fienberg, S.E.: Adaptive name matching in information integration. IEEE Intelligent Systems **18**(5), 16–23 (2003)
4. Binette, O., Steorts, R.C.: (almost) all of entity resolution. Science Advances **8**(12) (2022)
5. DeepSeek-AI: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)
6. Dubey, A., Jauhri, A., Pandey, A., et al.: The llama 3 herd of models. CoRR **abs/2407.21783** (2024)
7. Fan, M., Han, X., Chai, J.F., Chengliang, Tang, N., Li, G., Du, X.: Cost-effective in-context learning for entity resolution: A design space exploration. In: Proceedings of the 40th IEEE International Conference on Data Engineering. pp. 3696–3709 (2024)
8. Fellegi, I.P., Sunter, A.B.: A theory for record linkage. Journal of the American Statistical Association **64**(328), 1183–1210 (1969)
9. Fu, T., Gu, J., Li, Y., Qu, X., Cheng, Y.: Scaling reasoning, losing control: Evaluating instruction following in large reasoning models. arXiv preprint arXiv:2505.14810 (2025)
10. Getoor, L., Machanavajjhala, A.: Entity resolution: Theory, practice & open challenges. Proceedings of the VLDB Endowment **5**(12), 2018–2019 (2012)

11. Li, H., Feng, L., Li, S., Hao, F., Chen, J.Z., Song, Y., Chen, L.: On leveraging large language models for enhancing entity resolution. CoRR **abs/2401.03426** (2024)
12. Li, L., Li, J., Gao, H.: Rule-based method for entity resolution. IEEE Transactions on Knowledge and Data Engineering **27**(1), 250–263 (2015)
13. Li, Y., Li, J., Suhara, Y., Doan, A., Tan, W.C.: Deep entity matching with pre-trained language models. vol. 14, pp. 50–60 (2020)
14. Mudgal, S., Li, H., Rekatsinas, T., Doan, A., Park, Y., Karahalios, K., Kumar, A.: Deep learning for entity matching: A design space exploration. In: Proceedings of the 2018 International Conference on Management of Data. pp. 19–34 (2018)
15. Narayan, A., Chami, I., Orr, L., Ré, C.: Can foundation models wrangle your data? vol. 16, pp. 738–746 (2022)
16. Nikoletos, K., Papadakis, G., Mandilaras, G., Koumarelas, I., Tserpes, K.: Jedai^3: Beyond batch, blocking & cleaning for entity resolution. Information Systems **109**, 102093 (2022)
17. Papadakis, G., Ioannou, E., Niedermeier, C., Fankhauser, P., Nejdl, W.: Blocking and filtering techniques for entity resolution: A survey. In: ACM Computing Surveys. vol. 53, pp. 1–42 (2021)
18. Paulsen, T., Thirumuruganathan, S., Wang, H., Zhang, D.: Sparkly: A simple yet surprisingly strong tf/idf approach to entity linking. In: Proceedings of the ACM SIGMOD International Conference on Management of Data (2023)
19. Peeters, R., Bizer, C.: Using ChatGPT for entity matching. In: International Conference on AI in Finance (2023)
20. Peeters, R., Bizer, C.: Entity matching using large language models. In: Proceedings of the International Conference on Extending Database Technology (2025)
21. Qwen Team: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025)
22. Tu, J., Fan, J., Tang, N., Wang, P., Chai, C., Li, G., Fan, R., Du, X.: Unicorn: A unified multi-tasking model for supporting matching tasks in data integration. In: Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data (2023)
23. Wang, T., Chen, X., Lin, H., Chen, X., Han, X., Sun, L., Wang, H., Zeng, Z.: Match, compare, or select? an investigation of large language models for entity matching. In: Proceedings of the 31st International Conference on Computational Linguistics. pp. 96–109. Association for Computational Linguistics, Abu Dhabi, UAE (Jan 2025), `https://aclanthology.org/2025.coling-main.8/`

A Supplementary Results

Table 4 presents the complete ranking of all 46 model configurations.

Table 5 reports the invalid response rate for the 16 models (out of 46) that produced at least one unparseable response. The remaining 28 models have 0% invalid outputs under our parser and fallback rule.

Table 6 reports per-dataset F1 for selected models alongside the original ComEM result for GPT-4o Mini [23]. Our replication closely matches (85.9 vs. 86.4 mean F1), validating methodological faithfulness. Amazon-Google (AG) is consistently the hardest dataset; DBLP-ACM (DA) and IMDB-TMDB (IM) are easiest.

Table 4. Full compound strategy ranking (46 models). F1 = mean±std across 8 datasets; P/R are means; std omitted for space. Cost = total API-equivalent token cost. Claude models marked †. (t)=thinking enabled; (a)=adaptive thinking; (l)/(m)/(h)=low/medium/high reasoning effort.

#	Model	F1	P	R	$	#	Model	F1	P	R	$
1	gpt-5(m)	93.17±6.0	92.04	94.42	12.29	24	gpt-4o	89.58±6.1	89.43	89.87	3.10
2	gpt-5(l)	93.14±6.4	92.26	94.12	6.25	25	kimi-k2-0905	89.43±6.0	85.25	94.21	0.28
3	gpt-5.2(l)	92.75±5.9	90.58	95.12	3.43	26	gpt-5-nano(m)	89.34±6.2	84.86	94.42	0.92
4	gpt-5.2(h)	92.54±6.1	90.22	95.08	5.74	27	gpt-oss:20b(l)	88.74±7.7	88.56	89.04	0.08
5	gpt-5.2(m)	92.53±6.2	90.33	94.92	4.52	28	gpt-5-nano(l)	88.38±7.1	83.63	93.83	0.29
6	gpt-5(h)	92.14±6.8	92.66	91.71	24.50	29	qwen3-235b	86.85±6.3	80.54	94.54	0.08
7	gpt-oss:120b(h)	92.10±6.3	89.91	94.46	1.20	30	deepseek-v3.1(t)	86.52±8.2	88.68	84.83	3.37
8	kimi-k2-think	91.84±6.4	90.00	93.83	8.29	31	gpt-4o-mini	85.88±6.7	84.82	87.63	0.72
9	gpt-5-mini(l)	91.72±6.1	88.64	95.08	0.71	32	gpt-5-nano(h)	85.71±6.0	92.61	79.92	1.83
10	gpt-5-mini(h)	91.62±6.2	88.47	95.08	2.30	33	†claude-opus-4.6	85.18±7.1	78.77	92.88	13.24
11	gpt-5-mini(m)	91.46±6.4	88.21	95.04	1.28	34	gemma3:27b	83.86±5.7	75.71	94.12	0.08
12	gpt-oss:20b(h)	91.36±6.5	89.86	93.00	0.46	35	gpt-4.1-nano	83.49±8.4	82.29	85.21	0.08
13	minimax-m2.5	91.34±5.7	90.64	92.12	3.74	36	†claude-haiku-4.5	82.64±5.7	74.14	93.42	4.24
14	gpt-oss:120b(m)	91.26±7.2	90.29	92.33	0.38	37	†claude-sonnet-4.6(a)	82.55±6.3	74.16	93.12	11.18
15	†claude-haiku-4.5(t)	91.19±6.4	88.63	94.00	8.64	38	†claude-sonnet-4.6	82.48±6.8	74.24	92.83	11.20
16	glm-5(t)	91.02±6.2	92.83	89.29	12.62	39	qwen3-235b-think	81.76±11.5	91.05	74.96	2.71
17	gpt-oss:20b(m)	90.87±6.2	89.69	92.17	0.17	40	†claude-opus-4.5	80.53±6.5	71.55	92.13	22.32
18	gpt-4.1	90.36±5.6	86.15	95.12	1.27	41	†claude-opus-4.5(t)	80.28±6.9	71.19	92.08	57.07
19	gpt-4.1-mini	90.26±6.7	88.91	91.79	0.26	42	llama4:scout	79.89±6.0	70.71	91.83	0.33
20	†claude-opus-4.6(a)	90.13±6.4	86.13	94.62	19.05	43	glm-5	79.77±10.6	91.75	71.38	0.54
21	gpt-oss:120b(l)	90.11±7.9	90.04	90.25	0.17	44	†claude-sonnet-4.5	79.57±6.4	70.01	92.17	11.92
22	glm-4.7-flash	90.08±6.2	88.67	91.62	1.74	45	deepseek-v3.1	79.16±8.2	74.35	85.38	0.48
23	†claude-sonnet-4.5(t)	89.70±6.4	85.23	94.79	22.32	46	granite4:32b	78.49±9.5	81.89	77.08	0.09

Table 5. Invalid response rate for models with > 0% unparseable outputs. "Inv" is the count of invalid responses out of N=3,200 selecting-stage outputs (8 datasets × 400 anchors). (t)=thinking enabled; (l)/(m)/(h)=low/medium/high reasoning effort.

Model	Inv	%	Model	Inv	%
gpt-5-nano(h)	978	30.6	gpt-4.1-mini	41	1.3
glm-5	649	20.3	gpt-5-mini(h)	35	1.1
glm-5(t)	367	11.5	glm-4.7-flash	34	1.1
gpt-5(h)	185	5.8	kimi-k2-think	30	0.9
gpt-4o	148	4.6	gpt-oss:20b(h)	21	0.7
gpt-5-nano(m)	119	3.7	deepseek-v3.1	9	0.3
gpt-4.1-nano	83	2.6	gpt-5(m)	8	0.3
deepseek-v3.1(t)	41	1.3	gpt-oss:120b(h)	8	0.3

Table 6. Per-dataset F1 for selected models. AG is consistently hardest; DA and IM easiest. (t)=thinking enabled; (l)/(m)/(h)=low/medium/high reasoning effort. ⋆Original ComEM result from Wang et al. [23] for methodological validation.

Model	AB	AG	DA	DS	IM	IV	TT	WA	Avg
gpt-5(m)	96.56	80.52	98.68	95.25	98.35	90.70	90.13	95.17	93.17
gpt-5(l)	96.21	79.42	99.01	95.74	98.35	90.16	90.42	95.84	93.14
gpt-oss:120b(h)	94.49	78.37	98.36	94.36	96.60	90.82	88.89	94.88	92.10
kimi-k2-think	94.81	78.11	97.71	95.11	96.36	88.82	88.92	94.89	91.84
minimax-m2.5	94.53	79.48	96.89	93.85	95.45	88.15	88.71	93.68	91.34
claude-haiku-4.5(t)	94.47	76.97	96.61	92.89	96.73	90.20	88.26	93.40	91.19
gpt-4o	94.12	76.18	95.22	89.04	94.74	89.58	88.33	89.46	89.58
glm-5(t)	94.00	79.19	97.99	91.62	95.59	86.51	88.00	95.22	91.02
glm-4.7-flash	91.95	77.09	97.39	93.14	95.14	88.89	88.43	88.59	90.08
gpt-4o-mini	88.10	71.95	90.38	86.86	91.96	79.70	89.49	88.63	85.88
Original ComEM result [23]									
gpt-4o-mini⋆	88.24	71.47	90.58	87.84	95.62	78.07	90.97	88.56	86.42